Epidemic Simulation (SEIR Model) - Complete Walkthrough

Table of Contents

1. Project Overview
2. Dependencies and Imports
3. Simulation Parameters Explained
4. Code Structure and Classes
5. Main Simulation Loop
6. Key Algorithms and Behaviors
7. SEIR Disease Model
8. User Controls and Features
9. Data Visualization
10. Running the Simulation

---

1. Project Overview

What This Project Does

This is an **Advanced Epidemic Simulation** implementing the **SEIR (Susceptible-Exposed-Infected-Recovered) Disease Model** with:

- **Multiple Countries**: 4 regions with travel between them
- **Day/Night Cycle**: Agents have different behaviors during day and night
- **Age Demographics**: Young, Adult, and Elderly with different risk factors
- **Intervention Systems**: Quarantine, Safe Zones, Border Controls, Vaccination
- **Economy System**: Money management with bankruptcy consequences
- **Hospital Capacity**: Limited healthcare resources affecting death rates

Key Features

- **SEIR Model**: More realistic than SIR - includes incubation period
- **Spatial Dynamics**: Agents move between countries and gathering spots
- **Age-Based Risk**: Elderly are 5x more vulnerable, Young are more social
- **Economic Constraints**: Interventions cost money; bankruptcy disables services
- **Real-time Visualization**: Live population graphs and statistics
- **Interactive Controls**: Toggle interventions, vaccinate, adjust parameters

---

2. Dependencies and Imports

Lines 1-6: Import statements

```
import pygame          # Graphics and game loop

import random          # Random number generation

import math            # Mathematical functions (cos, sin, etc.)

import os              # File system operations

import matplotlib.pyplot as plt  # Graph generation

from collections import deque   # (Note: imported but not used in current
code)
```

- **pygame**: Handles all rendering, event processing, and the main game loop
- **random**: Used for random positions, travel decisions, infection chances, and demographics
- **math**: Used for trigonometry when calculating velocities from angles
- **matplotlib**: Generates high-quality graphs when saving scenarios

---

3. Simulation Parameters Explained

All parameters are defined at the top of the file (lines 8-75) for easy tuning.

Screen Configuration (Lines 12-14)

```
WIDTH, HEIGHT = 1000, 700  # Window size in pixels

FPS = 60                   # Frames per second (simulation speed)
```

- Larger window than Project One to accommodate 4 countries
- Same 60 FPS for smooth simulation

Visual Colors (Lines 16-27)

```
BG_COLOR_DAY = (20, 20, 20)        # Dark gray for daytime
```

```
BG_COLOR_NIGHT = (5, 5, 15)        # Very dark blue for nighttime

COLOR_SUSCEPTIBLE = (50, 150, 255)  # Blue - healthy, can get infected

COLOR_EXPOSED = (255, 200, 50)      # Yellow - incubating (not yet
infectious)

COLOR_INFECTED = (255, 50, 50)      # Red - actively spreading disease

COLOR_RECOVERED = (50, 200, 50)     # Green - immune

COLOR_DEAD = (100, 100, 100)        # Gray - deceased

COLOR_QUARANTINE = (50, 0, 0)       # Dark red background for isolation zone

COLOR_BORDER = (255, 255, 255)       # White borders

COLOR_HOSPITAL = (255, 100, 255)     # Pink/Purple for UI elements

COLOR_MONEY = (255, 215, 0)          # Gold for money display
```

- Color coding matches SEIR model states
- Day/night backgrounds create visual distinction

Population Constants (Lines 29-33)

```
POPULATION_SIZE = 400        # Total number of agents

INITIAL_INFECTED = 5         # Starting infected count

AGENT_RADIUS = 4             # Size of agent circles

AGENT_SPEED_BASE = 2.0       # Base movement speed (pixels per frame)
```

- **POPULATION_SIZE**: Total agents distributed across 4 countries
- **INITIAL_INFECTED**: Small initial outbreak to observe spread

Disease Parameters - SEIR Model (Lines 35-41)

```
INFECTION_RADIUS = 15                    # Distance for infection spread
(pixels)
```

```
INFECTION_PROBABILITY = 0.1                # 10% chance per frame when in
range

INCUBATION_PERIOD_FRAMES = 300             # ~5 seconds before becoming
infectious

RECOVERY_TIME_FRAMES = 600                 # ~10 seconds to recover

BASE_DEATH_CHANCE = 0.001                  # 0.1% chance per frame (normal
conditions)

OVERFLOW_DEATH_CHANCE = 0.01               # 1% chance per frame (hospital
full)
```

- **INFECTION_RADIUS**: Proximity required for transmission
- **INFECTION_PROBABILITY**: Base transmission rate (adjustable with UP/DOWN keys)
- **INCUBATION_PERIOD_FRAMES**: Time in "EXPOSED" state before becoming infectious
- **RECOVERY_TIME_FRAMES**: Time infected before recovery
- **Death Chances**: Age multipliers apply to these base values

Vaccination System (Lines 43-45)

```
VACCINATION_EFFICACY = 0.95               # 95% success rate

VACCINATION_WILLINGNESS = 0.8              # 80% of population willing to
vaccinate
```

- **EFFICACY**: Chance vaccine provides immunity
- **WILLINGNESS**: Percentage of agents that accept vaccination

Social, Travel, and Time (Lines 47-51)

```
TRAVEL_CHANCE = 0.002                      # 0.2% chance per frame to travel

SOCIAL_WEIGHT = 0.05                       # Strength of attraction to
work/school

DAY_LENGTH_FRAMES = 1200                   # 20 seconds per day (at 60 FPS)

NIGHT_START = 800                          # Night begins at frame 800 (66%
```

```
through day)
```

- **TRAVEL_CHANCE**: Probability of inter-country travel per frame
- **SOCIAL_WEIGHT**: How strongly agents are drawn to gathering spots
- **DAY_LENGTH_FRAMES**: Full day/night cycle duration
- **NIGHT_START**: When day transitions to night (18:00 equivalent)

Age Demographics (Lines 53-59)

```
AGE_STATS = {
    'YOUNG': (1.2, 0.1, 1.5),   # (Speed Mult, Death Mult, Social Mult)
    'ADULT': (1.0, 1.0, 1.0),   # Average baseline
    'ELDERLY': (0.6, 5.0, 0.5)  # Slow, Vulnerable, Isolated
}
```

- **YOUNG**: 20% faster, 10% death risk, 50% more social
- **ADULT**: Baseline (1.0x for all stats)
- **ELDERLY**: 40% slower, 5x death risk, 50% less social
- Distribution: 20% Young, 60% Adult, 20% Elderly

Intervention Defaults (Lines 61-66)

```
QUARANTINE_ENABLED = False                # Isolation zone for sick
SAFE_ZONE_ENABLED = False                 # Shelter for healthy
QUARANTINE_DETECTION_CHANCE = 0.05        # 5% chance to detect infected
QUARANTINE_EXPOSED_DETECTION_CHANCE = 0.02 # 2% chance to detect exposed
(contact tracing)
BORDERS_CLOSED = False                     # Travel restrictions
```

- All interventions start disabled
- Detection chances represent testing/contact tracing effectiveness

Economy & Hospital (Lines 68-75)

```
HOSPITAL_CAPACITY = 50                    # Max infected before overflow

STARTING_MONEY = 1000                     # Initial budget

INCOME_PER_HEALTHY = 0.1                  # Money per healthy agent per
frame

COST_QUARANTINE = 10                      # Cost per frame for isolation

COST_SAFE_ZONE = 10                       # Cost per frame for safe zone

COST_BORDERS = 15                         # Cost per frame for border
closure

COST_VACCINE = 20                         # Cost per vaccination
```

- **HOSPITAL_CAPACITY**: When exceeded, death rate increases 10x
- **INCOME_PER_HEALTHY**: Only Susceptible and Recovered generate income
- **Costs**: All interventions have ongoing or per-use costs

---

4. Code Structure and Classes

4.1 Initialization (Lines 77-86)

```
pygame.init()

screen = pygame.display.set_mode((WIDTH, HEIGHT))

pygame.display.set_caption("Epidemic Simulation: SEIR, Age & Day/Night")

clock = pygame.time.Clock()

font = pygame.font.SysFont(None, 24)

big_font = pygame.font.SysFont(None, 48)
```

- Initializes Pygame, creates the window, sets up the clock for FPS control
- Two fonts: regular (24px) and big (48px) for different UI elements

4.2 Country Class (Lines 92-109)

Purpose: Represents a geographic region with a gathering spot

Attributes:

```
self.name = name              # Country name (e.g., "North West")
self.rect = rect              # Pygame Rect defining boundaries
self.color = color            # Border color for visualization
self.gathering_spot = Vector2 # Work/School location (random within country)
```

Key Methods:

1. `__init__(name, rect, color)` (Lines 93-101):
   - Stores country properties
   - Creates a random gathering spot near the center
   - This is where agents go during the day
2. `draw(surface)` (Lines 103-109):
   - Draws country border as a rectangle
   - Displays country name
   - Draws gathering spot as a gray circle with "Work" label

Usage: 4 countries are created, dividing the screen into quadrants

4.3 Agent Class (Lines 111-250)

Purpose: Represents an individual person in the simulation with SEIR states

Attributes (Lines 112-139):

```
self.home_country = country           # Which country they belong to
self.home_pos = Vector2               # Where they sleep (random in country)
self.position = Vector2               # Current location
self.age_group = 'YOUNG'/'ADULT'/'ELDERLY'  # Demographics
```

```
self.speed_mult, self.death_mult, self.social_mult  # Age multipliers

self.velocity = Vector2                # Movement direction and speed

self.state = "SUSCEPTIBLE"             # SEIR state

self.days_exposed = 0                  # Frames in exposed state

self.days_infected = 0                 # Frames in infected state

self.immunity = 0.0                    # 0.0 to 1.0 (vaccination/recovery)

self.in_quarantine = False             # In isolation zone

self.in_safe_zone = False              # In safe zone

self.wants_vaccine = bool              # Willingness to vaccinate
```

Initialization (Lines 112-139):

- Randomly assigns age group (20% Young, 60% Adult, 20% Elderly)
- Sets multipliers based on age
- Starts as SUSCEPTIBLE with no immunity
- Random home position within country
- 80% chance to want vaccine (based on VACCINATION_WILLINGNESS)

Key Methods:

1. `move(bounds_rect, is_night)` (Lines 141-185):

Day Behavior (Lines 152-156):

```python
if not is_night:
    to_spot = (self.home_country.gathering_spot - self.position)
    target_vector += to_spot.normalize() SOCIAL_WEIGHT self.social_mult
```

- Agents are drawn toward their country's gathering spot

- Social multiplier makes Young more social, Elderly less social

- Creates clustering during day (work/school simulation)

Night Behavior (Lines 147-151):

```python
if is_night:
    to_home = (self.home_pos - self.position)
    if to_home.length() > 5:
        target_vector += to_home.normalize() * 0.1
```

- Strong pull toward home position
- Agents return home at night (social distancing)

Random Wander (Lines 160-163):

```python
if random.random() < 0.1:
    angle = random.uniform(-0.5, 0.5)
    self.velocity = self.velocity.rotate_rad(angle)
```

- 10% chance per frame to add random rotation
- Prevents perfectly predictable movement

Speed Control (Lines 165-168):

```python
speed = AGENT_SPEED_BASE * self.speed_mult
if self.velocity.length() > speed:
    self.velocity = self.velocity.normalize() * speed
```

- Caps velocity at age-appropriate speed
- Young move faster, Elderly slower

Boundary Bouncing (Lines 172-185):

- Agents bounce off country/zone boundaries

- Prevents agents from leaving their designated area

2. `update_health(hospital_overflow)` (Lines 187-212):

Exposed → Infected Transition (Lines 189-193):

```python
if self.state == "EXPOSED":
    self.days_exposed += 1
    if self.days_exposed > INCUBATION_PERIOD_FRAMES:
        self.state = "INFECTED"
```

- After 300 frames (~5 seconds), exposed becomes infectious
- This is the "E" stage of SEIR

Infected → Recovered/Dead (Lines 196-212):

```python
elif self.state == "INFECTED":
    self.days_infected += 1
    # Death check
    base_chance = OVERFLOW_DEATH_CHANCE if hospital_overflow else BASE_DEATH_CHANCE
    chance = base_chance * self.death_mult
    if random.random() < chance:
        self.state = "DEAD"
        return
    # Recovery check
    if self.days_infected > RECOVERY_TIME_FRAMES:
        self.state = "RECOVERED"
        self.immunity = 1.0
```

- Death chance depends on hospital capacity and age

- Elderly have 5x higher death risk

- Hospital overflow increases death rate 10x

- After 600 frames (~10 seconds), infected recover

3. `travel(countries)` (Lines 214-231):

```python
if self.state == "DEAD" or self.in_quarantine or self.in_safe_zone: return
chance = TRAVEL_CHANCE
if BORDERS_CLOSED:
    chance *= 0.05  # 95% reduction, but 5% still travel (smugglers/leaks)
if random.random() < chance:
    dest = random.choice(countries)
    if dest != self.home_country:
        # Teleport to new country
```

- Dead, quarantined, or safe-zone agents cannot travel

- Border closure reduces travel by 95% (not 100% - realistic leak)

- Agents teleport to random position in destination country

4. `draw(surface)` (Lines 233-250):

- Color based on SEIR state:

  - Blue = Susceptible

  - Yellow = Exposed (incubating)

  - Red = Infected

  - Green = Recovered

  - Gray = Dead

- Size varies by age:

  - Young: radius - 1 (smaller)

- Adult: base radius
- Elderly: radius + 1 (larger)

4.4 Simulation Class (Lines 252-664)

Purpose: Main controller managing the entire epidemic simulation

Initialization (Lines 253-295):

```
def __init__(self):
    self.running = True
    self.paused = False
    self.time_of_day = 0  # 0 to DAY_LENGTH_FRAMES
    # Economy
    self.money = STARTING_MONEY
    # Create 4 countries (quadrants)
    self.countries = [
        Country("North West", pygame.Rect(0, 0, mid_x, mid_y), ...),
        Country("North East", pygame.Rect(mid_x, 0, mid_x, mid_y), ...),
        Country("South West", pygame.Rect(0, mid_y, mid_x, mid_y), ...),
        Country("South East", pygame.Rect(mid_x, mid_y, mid_x, mid_y), ...)
    ]
    # Create zones
    self.quarantine_rect = pygame.Rect(...)  # Right side
    self.safe_zone_rect = pygame.Rect(...)   # Left side
    # Create 400 agents
    self.agents = [Agent(random.choice(self.countries)) for _ in range(400)]
    # Infect 5 initial agents
```

```
    for _ in range(INITIAL_INFECTED):
        random.choice(self.agents).state = "INFECTED"
    # History tracking
    self.history_susceptible = []
    self.history_exposed = []
    self.history_infected = []
    self.history_recovered = []
    self.history_dead = []
    self.history_new_infections = []
```

Key Methods:

1. `handle_input()` (Lines 297-323):

- **SPACE**: Pause/Resume
- **B**: Toggle border closure
- **Q**: Toggle quarantine/isolation
- **S**: Toggle safe zone
- **V**: Vaccinate population (10% of susceptibles)
- **R**: Reset simulation
- **UP/DOWN**: Adjust infection probability
- **1/2**: Save graphs with scenario names

2. `vaccinate_population()` (Lines 325-338):

```python
for agent in self.agents:
    if agent.state == "SUSCEPTIBLE":
        if random.random() < 0.1:  # 10% vaccination rate per call
            if agent.wants_vaccine:
```

```
            cost += COST_VACCINE
            if random.random() < VACCINATION_EFFICACY:
                agent.immunity = 1.0
                agent.state = "RECOVERED"
```

- Only vaccinates 10% of susceptibles per call
- Only agents who want vaccine are vaccinated
- 95% efficacy chance
- Costs $20 per successful vaccination

3. `update()` (Lines 340-444):

Time Cycle (Lines 344-346):

```python
self.time_of_day = (self.time_of_day + 1) % DAY_LENGTH_FRAMES
is_night = self.time_of_day > NIGHT_START
```

- Increments day/night cycle
- Night starts at frame 800 (66% through day)

Hospital Overflow Check (Lines 351-352):

```python
infected_agents = [a for a in self.agents if a.state == "INFECTED"]
hospital_overflow = len(infected_agents) > HOSPITAL_CAPACITY
```

- Checks if infected count exceeds capacity (50)
- Passed to `update_health()` to increase death rates

Economy & Bankruptcy (Lines 354-363):

```python
if self.money <= 0:
```

```
    # All services automatically disabled
    QUARANTINE_ENABLED = False
    SAFE_ZONE_ENABLED = False
    BORDERS_CLOSED = False
# Deduct intervention costs
if QUARANTINE_ENABLED: self.money -= COST_QUARANTINE
if SAFE_ZONE_ENABLED: self.money -= COST_SAFE_ZONE
if BORDERS_CLOSED: self.money -= COST_BORDERS
```

- Bankruptcy disables all interventions
- Each active intervention costs money per frame

Income Generation (Lines 367-374):

```python
if agent.state == "SUSCEPTIBLE" or agent.state == "RECOVERED":
    self.money += INCOME_PER_HEALTHY
```

- Only healthy agents generate income
- Sick/dead agents don't contribute to economy

Movement Updates (Lines 379-386):

```python
if agent.in_quarantine:
    agent.move(self.quarantine_rect, is_night)
elif agent.in_safe_zone:
    agent.move(self.safe_zone_rect, is_night)
else:
    agent.move(agent.home_country.rect, is_night)
    agent.travel(self.countries)
```

```

- Quarantined agents move within isolation zone

- Safe zone agents move within safe zone

- Normal agents move in their country and can travel

**Quarantine Detection** (Lines 392-409):

```python
if QUARANTINE_ENABLED and not agent.in_quarantine:
    if agent.state == "INFECTED":
        if random.random() < QUARANTINE_DETECTION_CHANCE:  # 5%
            agent.in_quarantine = True
            # Teleport to quarantine zone
    elif agent.state == "EXPOSED":
        if random.random() < QUARANTINE_EXPOSED_DETECTION_CHANCE:  # 2%
            agent.in_quarantine = True
```

- Represents testing and contact tracing

- Infected have 5% detection chance

- Exposed have 2% detection chance (contact tracing)

Safe Zone Evacuation (Lines 411-419):

```python
if SAFE_ZONE_ENABLED and agent.state == "SUSCEPTIBLE":
    if random.random() < 0.05:  # 5% gradual evacuation
        agent.in_safe_zone = True
```

- Healthy agents gradually move to safe zone

- 5% chance per frame for susceptibles

- Protects them from infection
```

Infection Spread (Lines 421-435):

```python
if agent.state == "SUSCEPTIBLE":
    for infector in infected_agents:
        # Check zone isolation
        if infector.in_quarantine and not agent.in_quarantine: continue
        if agent.in_quarantine and not infector.in_quarantine: continue
        if agent.in_safe_zone and not infector.in_safe_zone: continue
        dist = agent.position.distance_to(infector.position)
        if dist < INFECTION_RADIUS:
            chance = INFECTION_PROBABILITY * (1.0 - agent.immunity)
            if random.random() < chance:
                agent.state = "EXPOSED"
```

- Only **INFECTED** agents can spread (not EXPOSED)
- Quarantine zones prevent cross-contamination
- Safe zones protect healthy agents
- Immunity reduces infection chance
- Distance-based transmission (15 pixel radius)

4. `draw()` (Lines 446-477):

- Draws day/night background
- Draws countries with borders
- Draws quarantine and safe zones (if enabled)
- Draws all agents
- Draws UI overlay

5. `draw_ui()` (Lines 479-576):

- **Time Display**: Large "DAY" or "NIGHT" indicator

- **Hospital Bar**: Visual capacity indicator (green/yellow/red)
- **Money Display**: Large gold number, red if bankrupt
- **Statistics Panel**: SEIR counts
- **Controls Panel**: Status of all interventions
- **Real-time Graph**: Mini graph overlay

6. `draw_realtime_graph()` (Lines 578-620):
   - Shows last 500 frames
   - Displays Exposed (yellow), Infected (red), Dead (gray)
   - Auto-scales to population size
7. `save_graphs(filename_prefix)` (Lines 622-651):
   - Creates two subplots:
     - Full SEIR timeline
     - New infections rate (smoothed)
   - Saves to `Graphs_For_Project_Two/` directory
8. `run()` (Lines 653-661):
   - Main game loop
   - Handles input, updates simulation, draws frame
   - Runs at 60 FPS

---

5. Main Simulation Loop

The `run()` method (Lines 653-661) contains the main game loop:

```
while self.running:
    clock.tick(FPS)            # Maintain 60 FPS
    self.handle_input()        # Process keyboard events
    self.update()              # Update simulation state
```

```
    self.draw()                # Render frame
```

Update Loop Breakdown (from `update()` method):

1. **Time Progression**: Increment day/night cycle
2. **Hospital Check**: Count infected, check for overflow
3. **Economy Update**:

- Deduct intervention costs

- Generate income from healthy agents

- Handle bankruptcy

4. **Agent Updates** (for each agent):

- Count SEIR states

- Update movement (day/night behavior)

- Update health (SEIR transitions)

- Check quarantine detection

- Check safe zone evacuation

- Check infection spread

5. **History Recording**: Store all counts for graphs

---

6. Key Algorithms and Behaviors

6.1 SEIR State Machine

States and Transitions:

```
SUSCEPTIBLE → (Infection) → EXPOSED → (Incubation) → INFECTED →
(Recovery/Death) → RECOVERED/DEAD
```

1. **SUSCEPTIBLE → EXPOSED**:

- Triggered by proximity to INFECTED agent

- Chance = `INFECTION_PROBABILITY * (1.0 - immunity)`

- Only INFECTED can spread (not EXPOSED)

2. **EXPOSED → INFECTED**:

- Automatic after `INCUBATION_PERIOD_FRAMES` (300 frames)

- Agent becomes infectious

3. **INFECTED → RECOVERED**:

- Automatic after `RECOVERY_TIME_FRAMES` (600 frames)

- Agent gains full immunity

4. **INFECTED → DEAD**:

- Random chance per frame

- `chance = (BASE_DEATH_CHANCE or OVERFLOW_DEATH_CHANCE) * death_mult`

- Elderly have 5x higher risk

6.2 Day/Night Behavior

Day (frames 0-799):

- Agents drawn to country's gathering spot
- Social multiplier affects attraction strength
- Creates clustering (work/school simulation)

Night (frames 800-1199):

- Agents return home
- Strong pull toward home position
- Reduces social contact (social distancing)

Visual Indicator: Background color changes (dark gray → very dark blue)

6.3 Age-Based Risk System

Death Risk Calculation:

```
base_chance = OVERFLOW_DEATH_CHANCE if hospital_overflow else
BASE_DEATH_CHANCE

chance = base_chance * self.death_mult
```

Examples:

- **Young**: 0.001 × 0.1 = 0.0001 (0.01% per frame)
- **Adult**: 0.001 × 1.0 = 0.001 (0.1% per frame)
- **Elderly**: 0.001 × 5.0 = 0.005 (0.5% per frame)
- **Elderly (Overflow)**: 0.01 × 5.0 = 0.05 (5% per frame!)

6.4 Zone Isolation Logic

Quarantine Zone:

- Isolated agents cannot spread to outside
- Outside agents cannot spread to isolated
- Represents perfect isolation (idealized)

Safe Zone:

- Healthy agents protected from outside infection
- If infected somehow enters, they spread inside
- Represents shelter-in-place

Cross-Contamination Prevention:

```
if infector.in_quarantine and not agent.in_quarantine: continue

if agent.in_quarantine and not infector.in_quarantine: continue

if agent.in_safe_zone and not infector.in_safe_zone: continue
```

6.5 Economy System

Income:

- `INCOME_PER_HEALTHY` per Susceptible/Recovered agent per frame
- At 60 FPS: 400 healthy agents × 0.1 = $40/second = $2400/minute

Expenses:

- Quarantine: $10/frame = $600/second
- Safe Zone: $10/frame = $600/second
- Borders: $15/frame = $900/second
- Vaccination: $20 per dose (one-time)

Bankruptcy:

- When money ≤ 0, all services automatically disabled
- No way to re-enable without resetting

---

7. SEIR Disease Model

7.1 What is SEIR?

SEIR extends the simpler **SIR** model by adding an **Exposed (E)** state:

- **S (Susceptible)**: Can be infected
- **E (Exposed)**: Infected but not yet infectious (incubation)
- **I (Infected)**: Actively spreading disease
- **R (Recovered)**: Immune (or Dead)

7.2 Why SEIR is More Realistic

SIR Model:

- Assumes immediate infectiousness upon infection
- Unrealistic for diseases with incubation periods

SEIR Model:

- Includes incubation period (E state)
- More accurately models diseases like COVID-19, influenza
- Allows for contact tracing (detecting E state)

7.3 Implementation Details

Exposed State:

- Agents in E state **cannot spread disease**
- Only visible through contact tracing (2% detection)
- After 300 frames, automatically becomes INFECTED

Infected State:

- Only I state agents can spread
- Checked every frame for nearby susceptibles
- Distance-based transmission (15 pixel radius)

Recovery:

- After 600 frames, infected recover
- Gain full immunity (immunity = 1.0)

- Cannot be reinfected

---

8. User Controls and Features

8.1 Complete Control Reference

| Key | Function | Effect | Cost |

|-----|----------|--------|------|

| **SPACE** | Pause/Resume | Freezes simulation | Free |

| **B** | Toggle Borders | Closes/opens travel | -$15/frame |

| **Q** | Toggle Quarantine | Enables/disables isolation | -$10/frame |

| **S** | Toggle Safe Zone | Enables/disables shelter | -$10/frame |

| **V** | Vaccinate | Vaccinates 10% susceptibles | -$20/dose |

| **R** | Reset | Restarts simulation | Free |

| **UP** | Increase Infection | +0.01 to probability | Free |

| **DOWN** | Decrease Infection | -0.01 to probability | Free |

| **1** | Save Extinction | Saves graph as scenario | Free |

| **2** | Save Survival | Saves graph as scenario | Free |

8.2 Intervention Strategies

Early Intervention:

- Close borders immediately
- Activate quarantine as soon as cases detected
- Vaccinate early (before outbreak spreads)

Balanced Approach:

- Monitor hospital capacity
- Activate interventions when cases rise
- Manage economy carefully

Late Intervention:

- Let disease spread, then react
- Risk: Hospital overflow, high death rate

- May be too late to control

8.3 Economic Management

Tips:

- Start with high money ($1000)
- Only activate needed interventions
- Monitor money closely (top-right display)
- Bankruptcy disables everything!

Income Sources:

- Healthy agents generate income
- More healthy = more money
- Sick/dead agents don't contribute

---

9. Data Visualization

9.1 Real-time Graph Overlay

Location: Bottom-right corner

Shows:

- **Yellow Line**: Exposed (incubating)
- **Red Line**: Infected (active cases)
- **Gray Line**: Dead (cumulative deaths)

Features:

- Last 500 frames (sliding window)
- Auto-scales to population size
- Updates every frame

9.2 Statistics Panel

Location: Bottom-left corner

Displays:

- Current SEIR counts
- Intervention status
- Control instructions

9.3 Hospital Capacity Bar

Location: Top-left

Visual:

- Green: < 80% capacity
- Yellow/Red: > 80% capacity
- Red text warning if overflow

9.4 Saved Graphs

Location: `Graphs_For_Project_Two/`

Content:

1. **Full SEIR Timeline**: All 5 states over entire simulation
2. **Infection Rate**: New cases per frame (smoothed with 10-frame rolling average)

File Names:

- `Scenario_1_Extinction.png` (Key 1)
- `Scenario_2_Survival.png` (Key 2)
- `epidemic_results.png` (default)

---

10. Running the Simulation

10.1 Starting the Simulation

```bash
python Project_Two.py
```

The simulation window will open with:

- 4 countries (quadrants)
- 400 agents (100 per country, approximately)
- 5 initially infected agents
- $1000 starting money
- Day/night cycle running

10.2 Visual Indicators

Agent Colors:

- **Blue Circles**: Susceptible (healthy, can get infected)
- **Yellow Circles**: Exposed (incubating, not yet infectious)
- **Red Circles**: Infected (actively spreading)
- **Green Circles**: Recovered (immune)
- **Gray Circles**: Dead

Agent Sizes:

- **Small**: Young (radius - 1)
- **Medium**: Adult (base radius)
- **Large**: Elderly (radius + 1)

Zones:

- **Red Rectangle (Right)**: Quarantine/Isolation zone
- **Blue Rectangle (Left)**: Safe Zone
- **Gray Circles**: Work/School gathering spots

Background:

- **Dark Gray**: Daytime
- **Very Dark Blue**: Nighttime

10.3 Understanding Epidemic Dynamics

Typical Progression:

1. **Initial Outbreak** (Frames 0-500):

- 5 infected agents start spreading

- Cases increase exponentially

- Exposed count rises first

2. **Peak Infection** (Frames 500-2000):

- Infected count peaks

- Hospital may overflow

- Death rate increases

- Recovered count starts rising

3. **Recovery Phase** (Frames 2000+):

- Infected count decreases

- Recovered count increases

- Susceptible count decreases

- May reach herd immunity

Possible Outcomes:

- **Herd Immunity**: Enough recovered to stop spread
- **Extinction**: All agents die (rare with default settings)
- **Oscillation**: Disease cycles if reintroduced
- **Containment**: Interventions successfully control spread

10.4 Experimentation Tips

1. **Test Interventions**:

- Try closing borders early

- Compare quarantine vs. safe zone

- Test vaccination timing

2. **Monitor Hospital**:

- Watch for overflow (red warning)

- Death rate increases 10x when full

- Elderly most at risk

3. **Economic Balance**:

- Don't activate all interventions at once

- Monitor money (top-right)

- Bankruptcy = game over for interventions

4. **Parameter Tuning**:

- Use UP/DOWN to adjust infection rate

- Observe how it affects spread speed

- Higher = faster spread, harder to control

5. **Age Demographics**:

- Watch elderly agents (larger circles)

- They die more often, especially during overflow

- Young agents are more social (spread faster)

10.5 Common Scenarios

Scenario 1: Extinction (Key 1):

- No interventions
- High infection rate
- Hospital overflow
- High death rate, especially elderly

Scenario 2: Survival (Key 2):

- Early interventions
- Border closure
- Quarantine activated
- Vaccination campaign
- Controlled spread, lower deaths

---

11. Mathematical Concepts

11.1 SEIR Model Equations

Differential Equations (Continuous):

```
dS/dt = −βSI/N

dE/dt = βSI/N − σE

dI/dt = σE − γI − μI

dR/dt = γI
```

Where:

- **β**: Transmission rate (INFECTION_PROBABILITY in code)
- **σ**: Incubation rate (1/INCUBATION_PERIOD_FRAMES)
- **γ**: Recovery rate (1/RECOVERY_TIME_FRAMES)
- **μ**: Death rate (BASE_DEATH_CHANCE or OVERFLOW_DEATH_CHANCE)
- **N**: Total population

Discrete Implementation (Code):

- Uses frame-based updates instead of continuous
- Probabilistic transitions instead of deterministic
- Spatial distance affects transmission

11.2 Infection Probability

```
chance = INFECTION_PROBABILITY * (1.0 - agent.immunity)
```

- Base probability: 0.1 (10% per frame when in range)
- Immunity reduces chance linearly
- Full immunity (1.0) = 0% chance

11.3 Death Probability

```
base_chance = OVERFLOW_DEATH_CHANCE if hospital_overflow else
BASE_DEATH_CHANCE

chance = base_chance * self.death_mult
```

- Base: 0.001 (normal) or 0.01 (overflow)
- Age multiplier: 0.1 (Young), 1.0 (Adult), 5.0 (Elderly)
- Per-frame probability (not cumulative)

11.4 Economic Model

Income:

```
income_per_frame = (susceptible_count + recovered_count) ×
INCOME_PER_HEALTHY
```

Expenses:

```
expenses_per_frame = (quarantine_cost × quarantine_enabled) +

                     (safe_zone_cost × safe_zone_enabled) +
```

```
                    (borders_cost × borders_closed)
```

Net:

```
money_change = income_per_frame - expenses_per_frame
```

---

12. Code Flow Summary

Execution Order (Per Frame)

1. **Event Processing**: Handle keyboard input
2. **Time Update**: Increment day/night cycle
3. **Hospital Check**: Count infected, check overflow
4. **Economy Update**:

- Deduct intervention costs

- Generate income

- Handle bankruptcy

5. **Agent Updates** (for each agent):

- Count SEIR states

- Update movement (day/night behavior)

- Update health (SEIR transitions)

- Check quarantine detection

- Check safe zone evacuation

- Check infection spread

6. **History Recording**: Store all counts
7. **Rendering**: Draw countries, zones, agents, UI, graphs
8. **Display**: Update screen

Key Design Patterns

- **State Machine**: SEIR states with automatic transitions
- **Spatial Simulation**: Distance-based infection spread

- **Economic Constraints**: Money limits intervention options
- **Age-Based Risk**: Demographics affect behavior and outcomes
- **Day/Night Cycle**: Temporal behavior changes
- **Zone Isolation**: Geographic separation prevents spread

Conclusion

This simulation demonstrates a complex **SEIR epidemic model** with:

- Realistic disease progression (incubation period)
- Multiple intervention strategies
- Economic constraints
- Age-based risk factors
- Spatial dynamics (countries, zones, travel)
- Temporal dynamics (day/night cycles)

The code is structured for easy modification and experimentation, making it an excellent learning tool for understanding:

- Epidemiological modeling
- Public health interventions
- Economic trade-offs in crisis management
- Agent-based simulation
- Complex systems with emergent behavior

File: `Project_Two.py`

Total Lines: 665

Classes: 3 (Country, Agent, Simulation)

Key Models: SEIR Disease Model, Economic System, Age Demographics

Interventions: Quarantine, Safe Zones, Border Control, Vaccination